

3008ICT Deep Learning

Week 7. LSTM, BRNN and Word Embedding

- **LSTM**
- **Bidirectional RNN (BRNN)**
- **Word embedding**

Adapted from slides at d2l.ai and Stanford CS224n

Notices

1. Response to student feedback:

- Workload too heavy: will be addressed
- AI-generated videos

2. Reminder: LLMs are not banned in this course. That is, you can use them in your study, but don't use LLMs for generating whole solutions for your assignments or exercises.

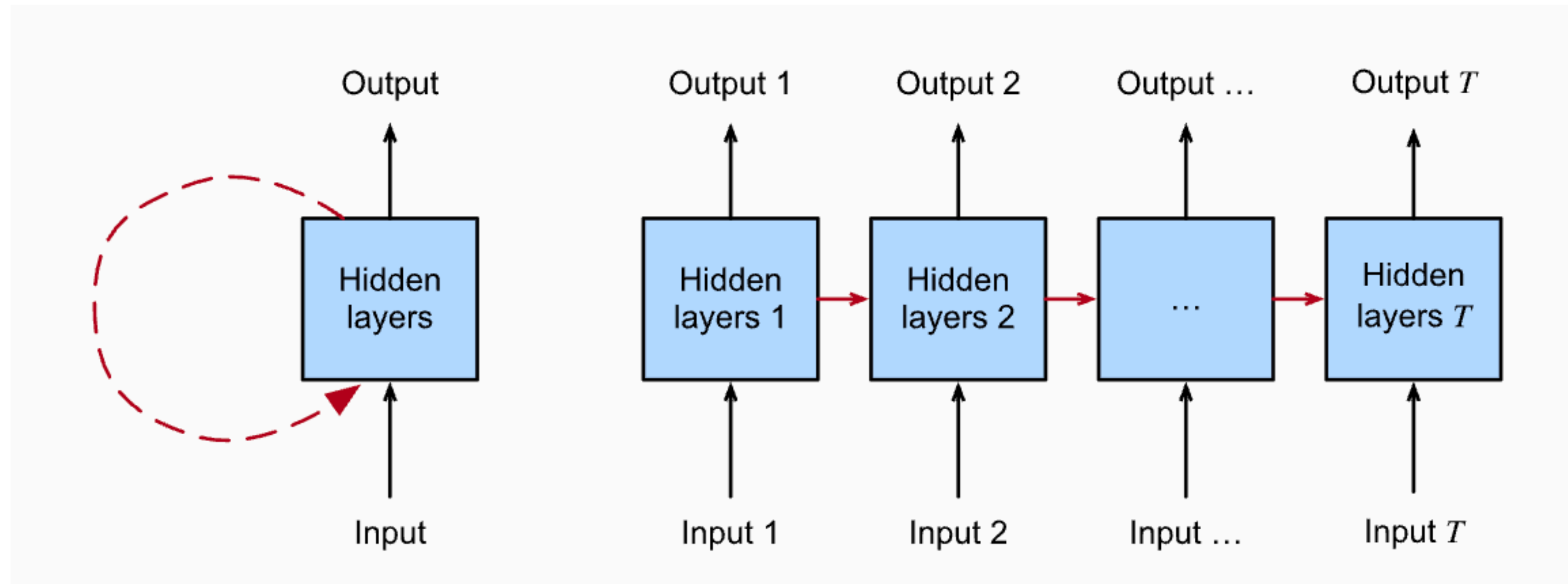
3. We are trying to get the marking for A1 done in two weeks, but could be longer.

4. Assignment 2 will be released by Monday, 27 April.

5. Make sure to practice on the homework exercises (prepare your final exam early).

Recap: Recurrent neural networks (RNNs)

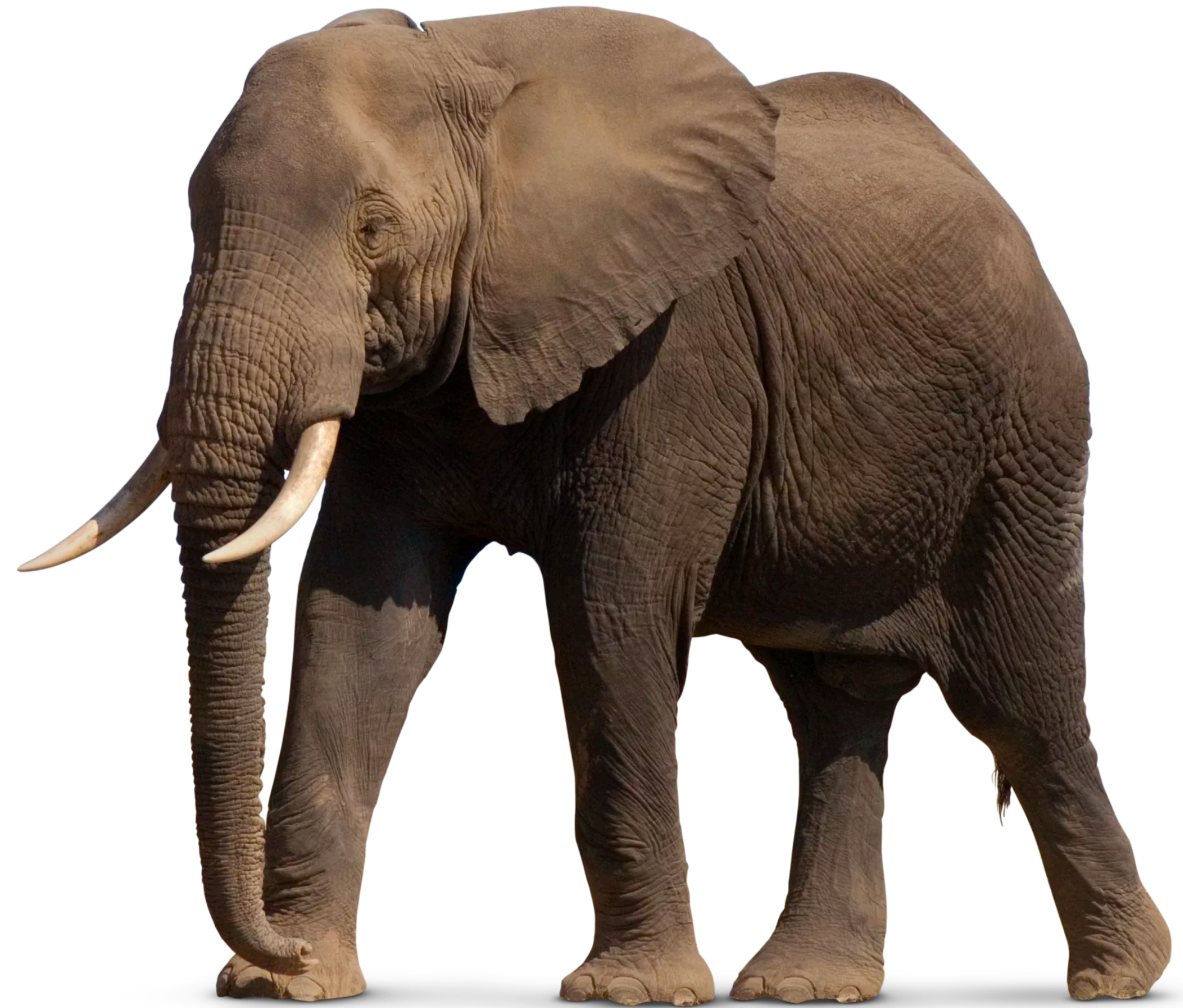
- RNNs are DL models capturing the dynamics of sequences via *recurrent* connection (cycles in the network of nodes).
- However, recurrent edges are defined in a precise way so that no ambiguity can arise.



Recap: How to fix the vanishing gradient problem?

- The main problem is that it's too difficult for the RNN to learn to preserve information over many time steps.
- In some RNNs, the hidden state is constantly being **rewritten**.
- First: How about an RNN with separate **memory** which is added to?
 - Long Short-Term Memory (LSTM) [[link](#)]
- And then: Creating more direct and linear pass-through connections in model
 - Attention, residual connections, etc.

Long Short Term Memory (LSTM)



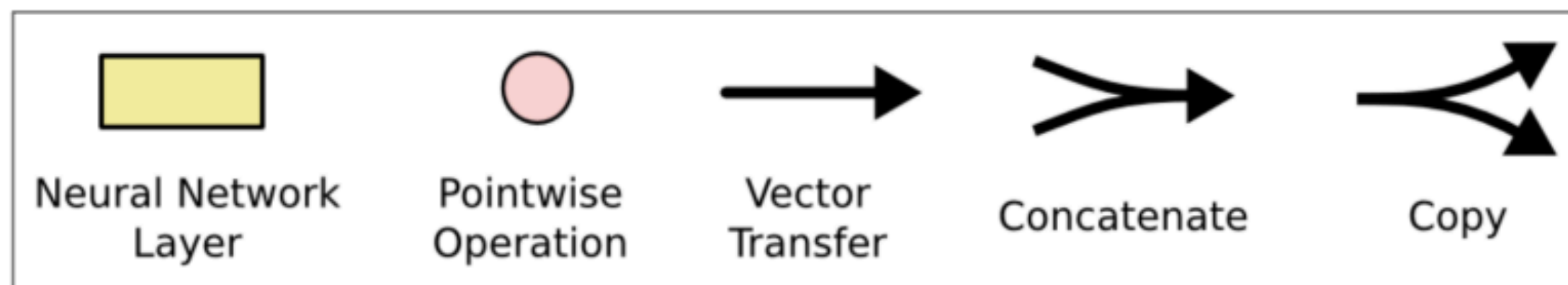
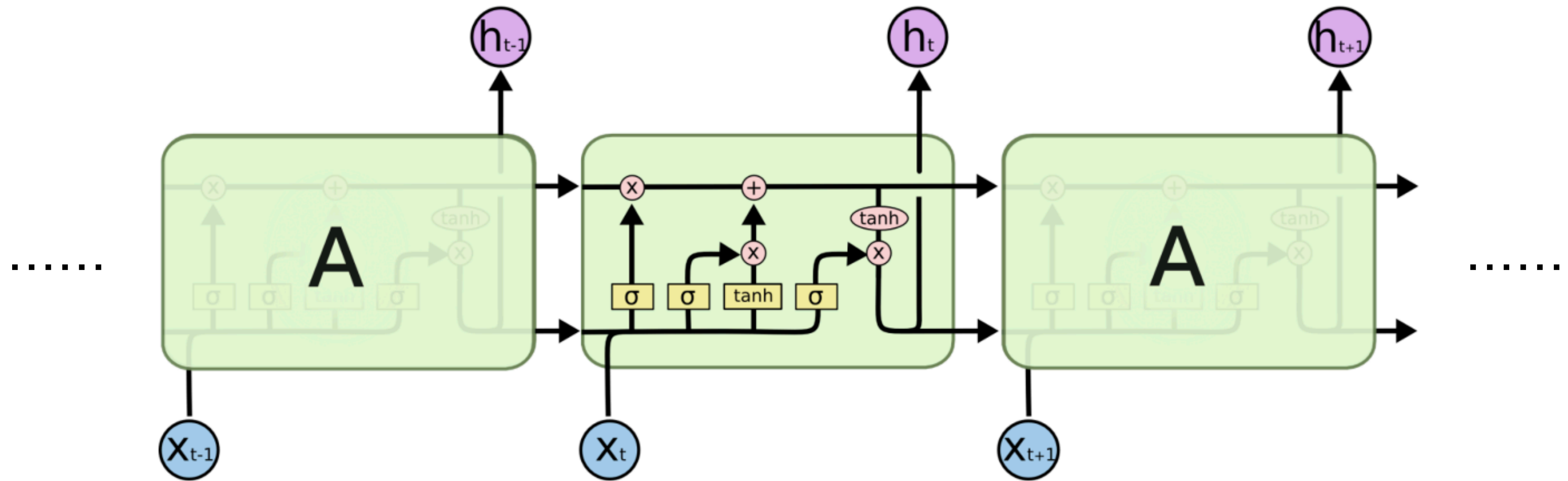
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short-Term Memory RNNs (LSTMs)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
 - Everyone cites that paper but really a crucial part of the modern LSTM is from Gers et al. (2000) 
- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors length n
 - The cell stores **long-term information**
 - The LSTM can **read**, **erase**, and **write** information from the cell
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
 - The gates are **dynamic**: their value is computed based on the current context

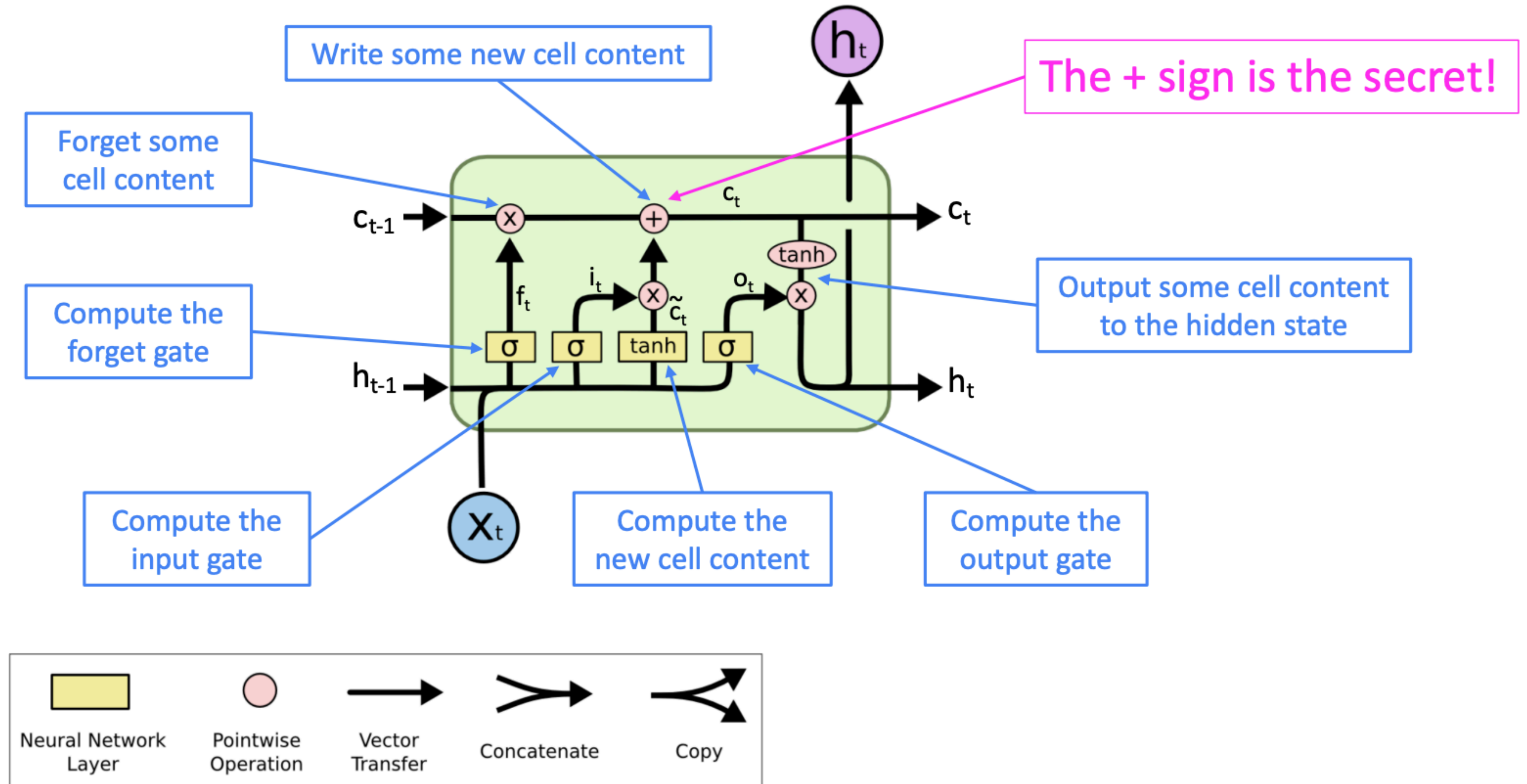
Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTM)

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$\mathbf{f}^{(t)} = \sigma \left(\mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f \right)$$

$$\mathbf{i}^{(t)} = \sigma \left(\mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i \right)$$

$$\mathbf{o}^{(t)} = \sigma \left(\mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o \right)$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh \left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c \right)$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)}$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh \mathbf{c}^{(t)}$$

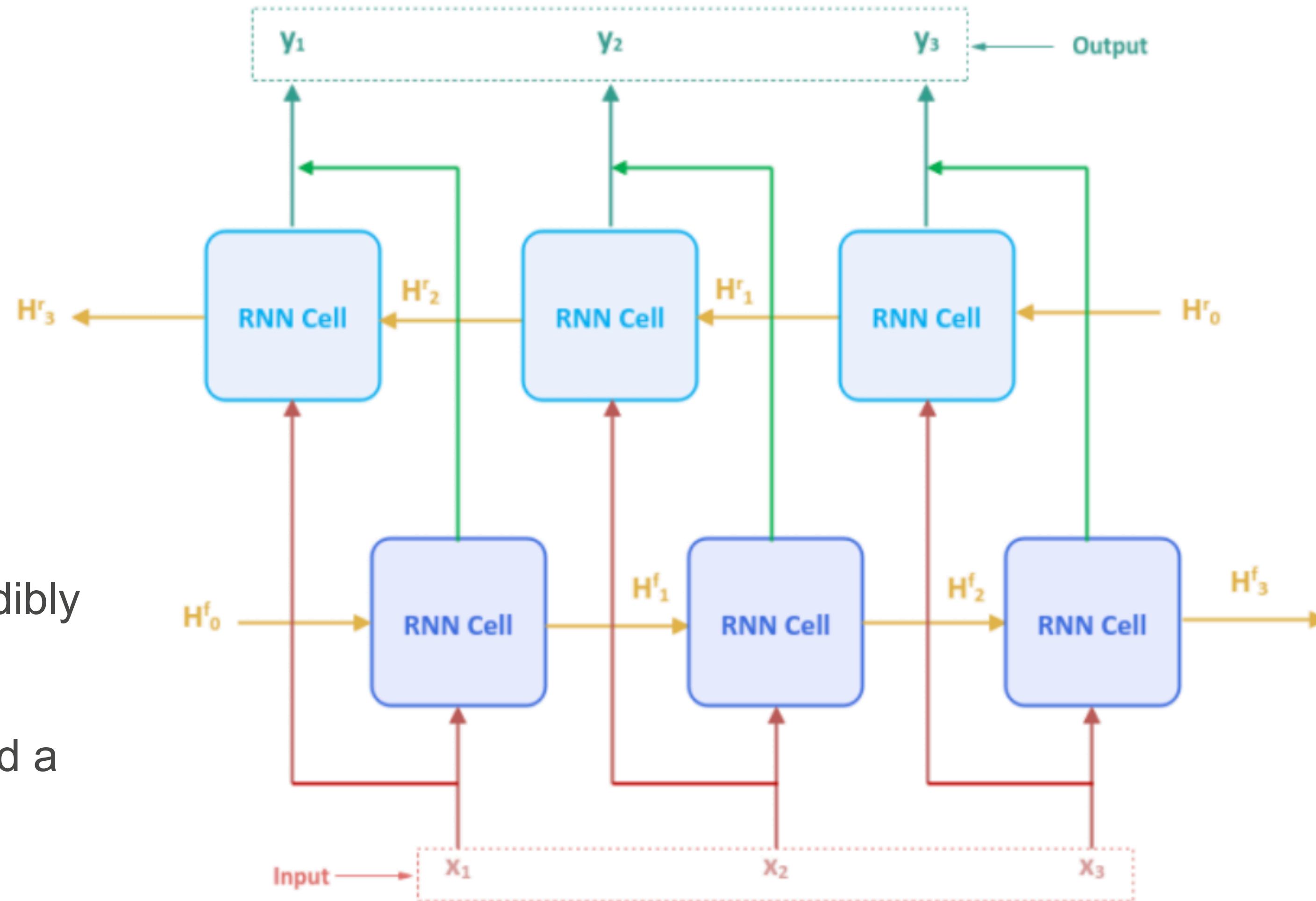
All these are vectors of same length n

Gates are applied using element-wise (or Hadamard) product: $\odot$

LSTMs: real-world success

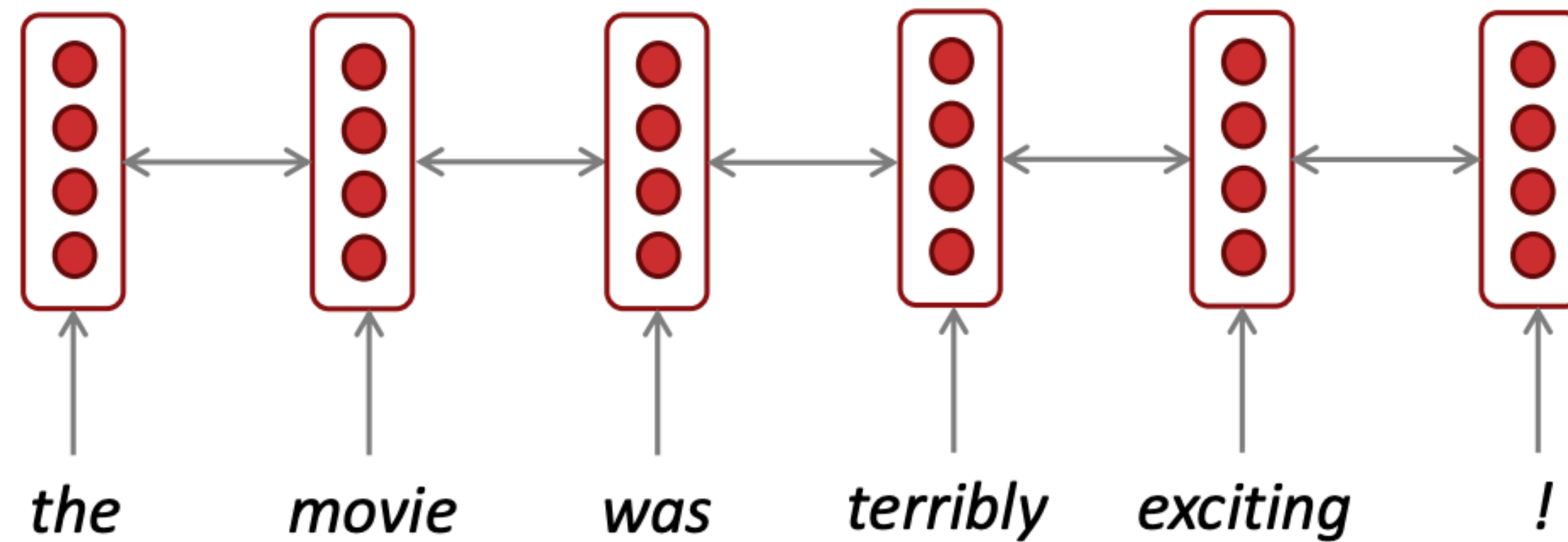
- In 2013–2015, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks
- Now, other approaches (e.g., Transformers) have become dominant for many tasks

Bidirectional Recurrent Neural Networks (BRNN)



- "The Apple was incredibly crisp and delicious."
- "The Apple announced a new product today."

Bidirectional RNNs: simplified diagram



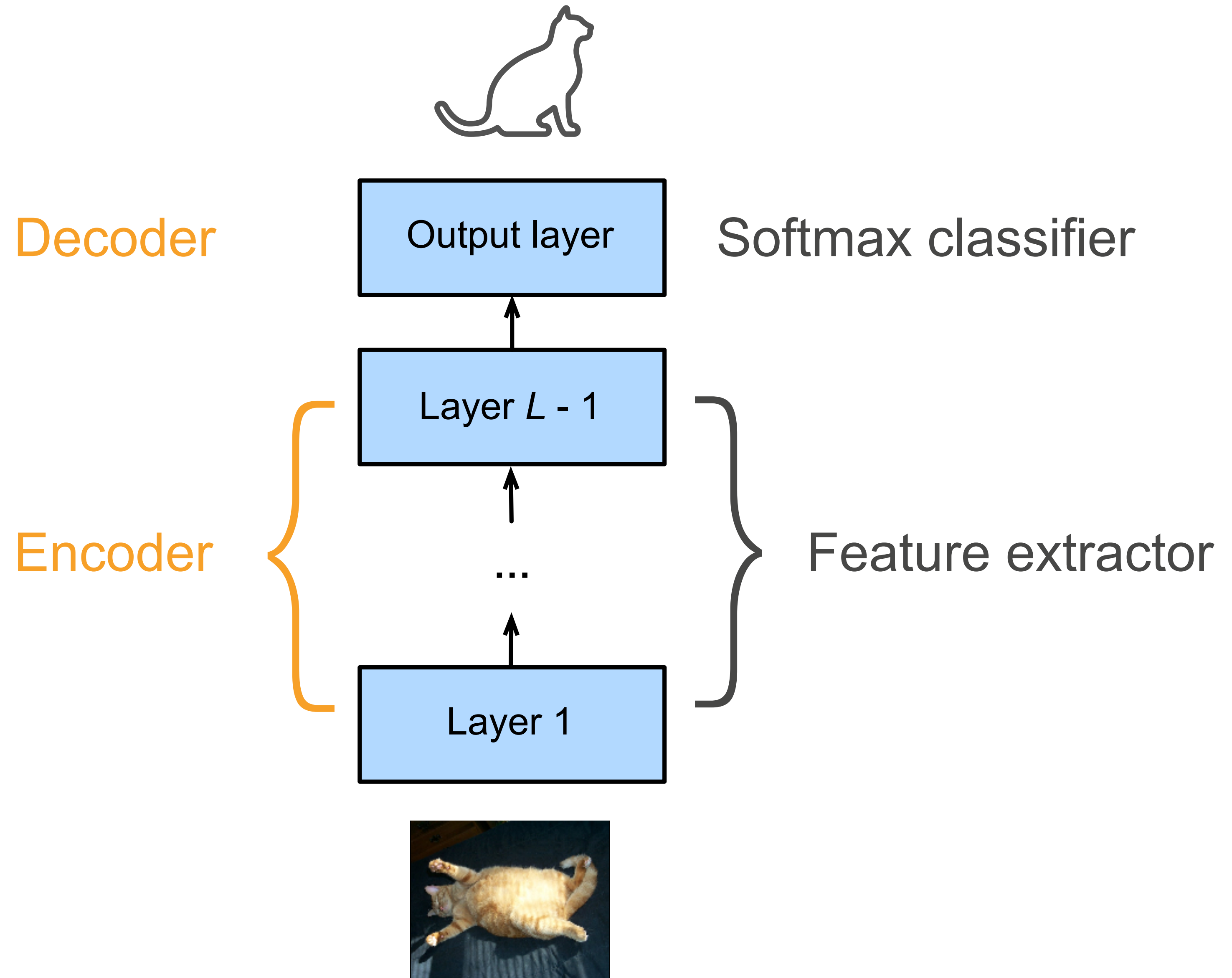
The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states

Bidirectional RNNs

- Note: bidirectional RNNs are only applicable if you have access to the **entire input sequence**
 - They are **not** applicable to Language Modeling, because in LM you *only* have left context available.
- If you do have entire input sequence (e.g., any kind of encoding), **bidirectionality is powerful** (you should use it by default).
- For example, **BERT** (**Bidirectional** Encoder Representations from Transformers) is a powerful pretrained contextual representation system **built on bidirectionality**.
 - You will learn more about **transformers** include BERT in a couple of weeks!

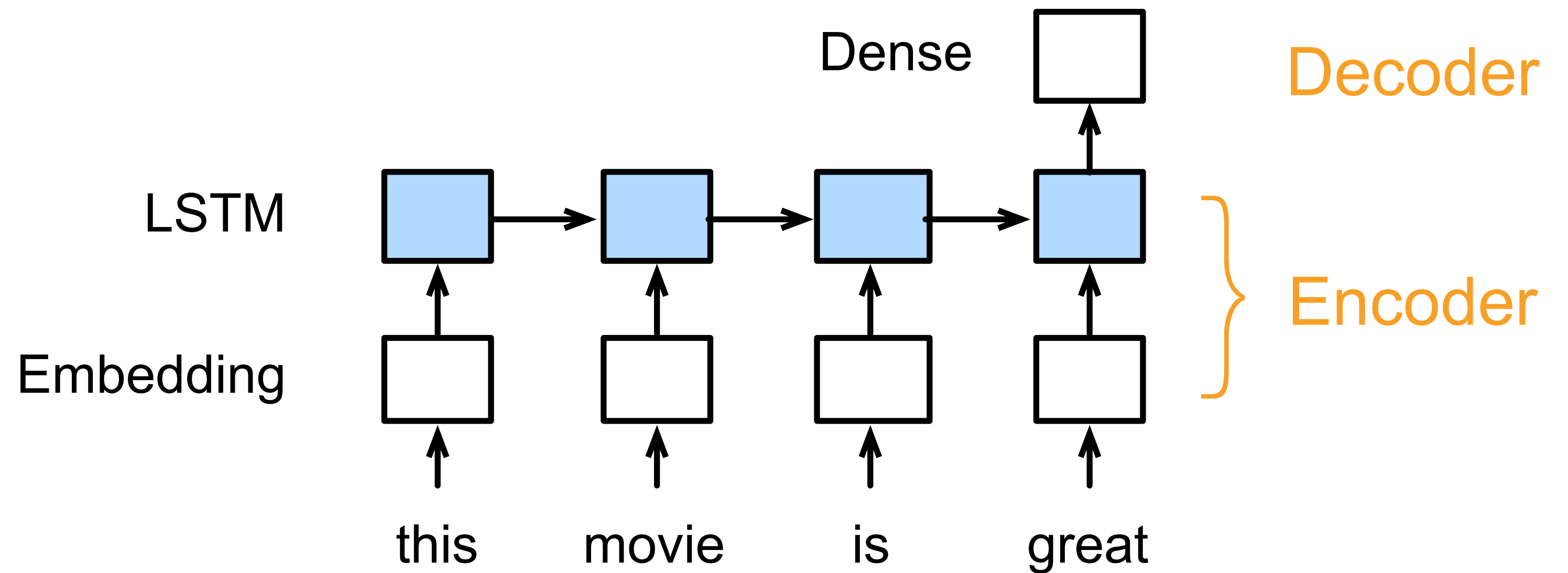
Rethink about CNN

- Encoder: encode inputs into intermediate presentation (features)
- Decoder: decode the presentation into outputs



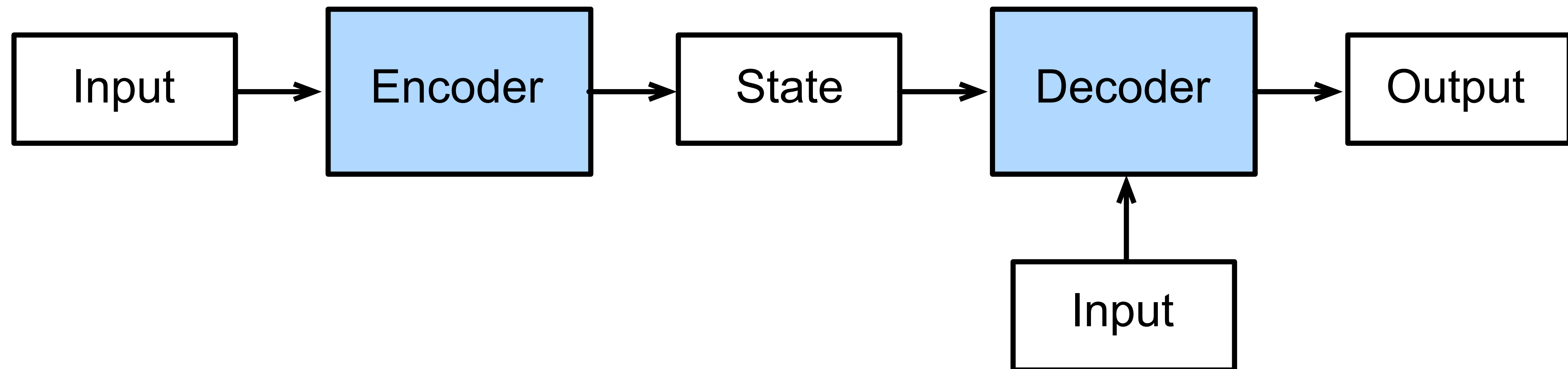
Rethink about RNN

- Encoder: present a piece of text as a vector
- Decoder: decode the presentation into outputs



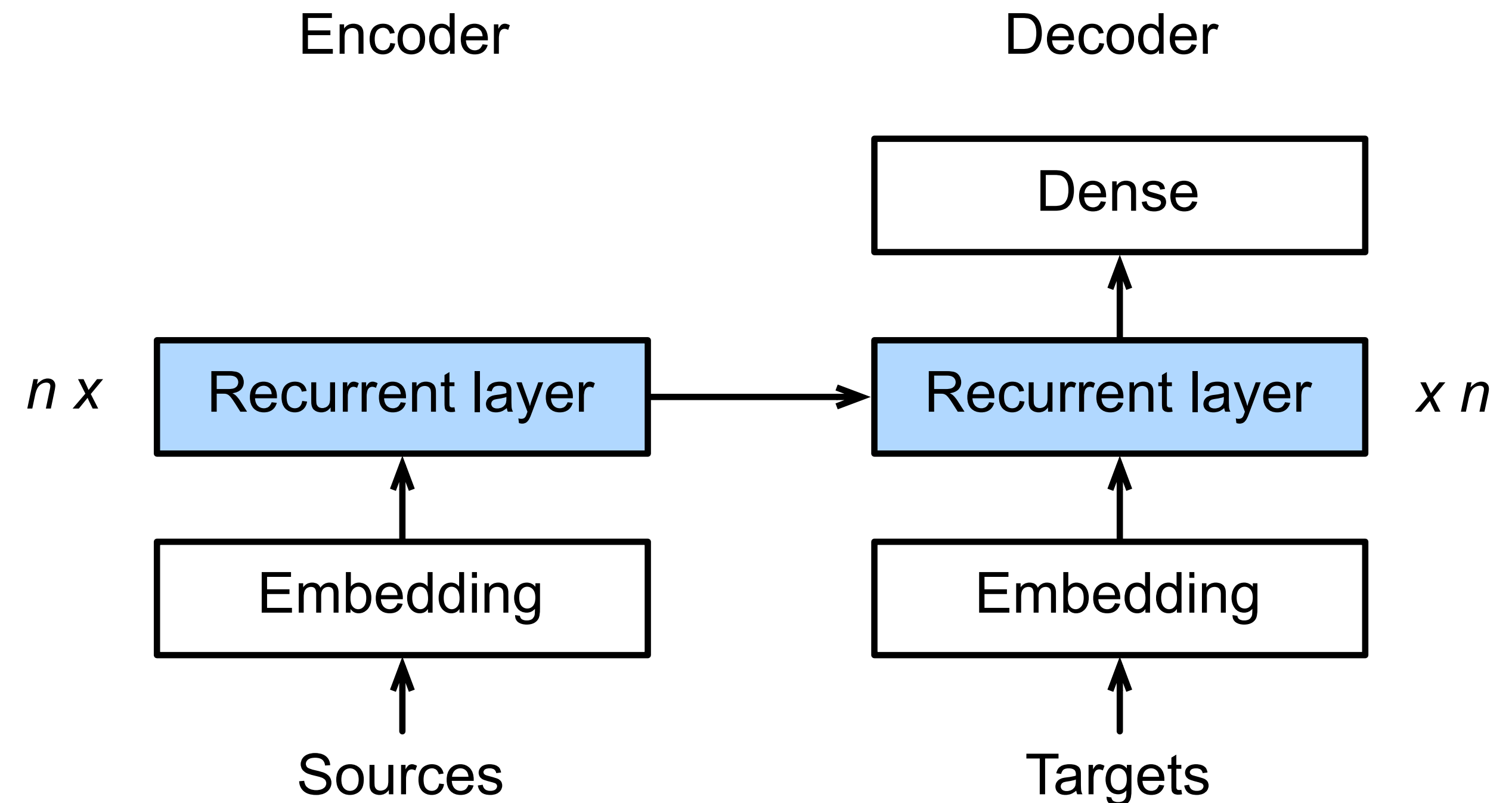
The Encoder-decoder Architecture

- A model is partitioned into two parts
 - The encoder process inputs
 - The decoder generates outputs



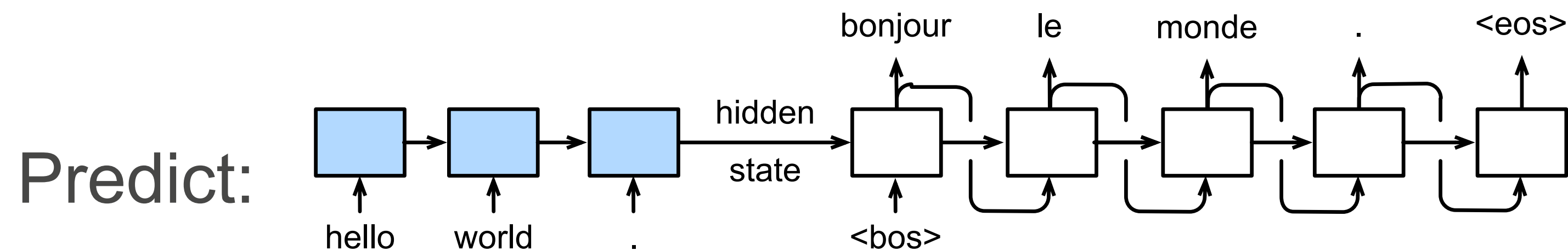
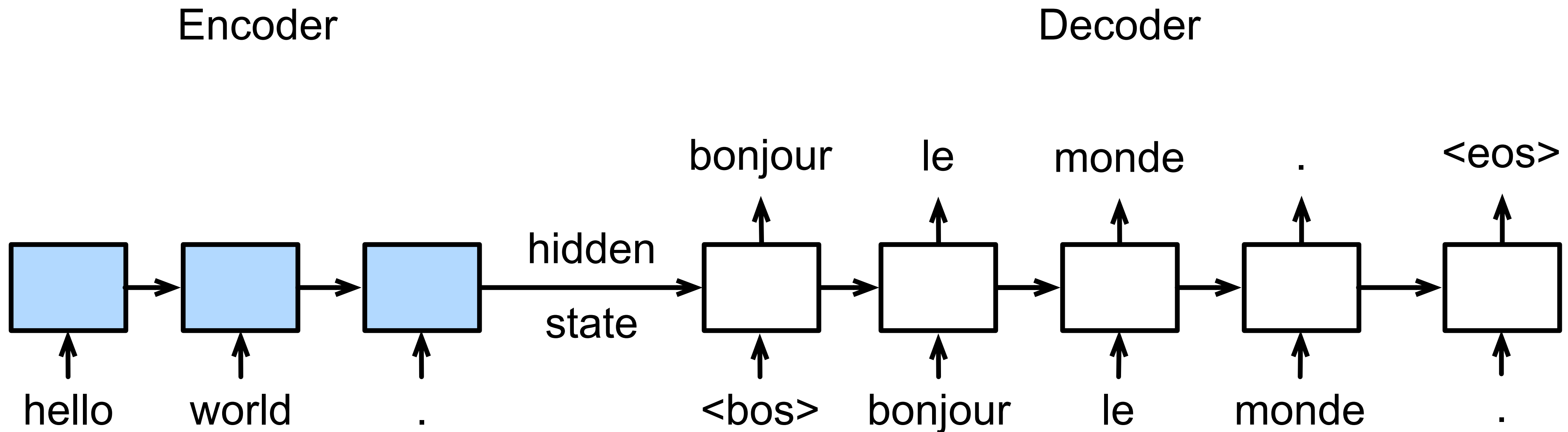
Encoder/Decoder Details

- The encoder is a standard RNN model without the output layer
- The encoder's hidden state in last time step is used as the decoder's initial hidden state



Training

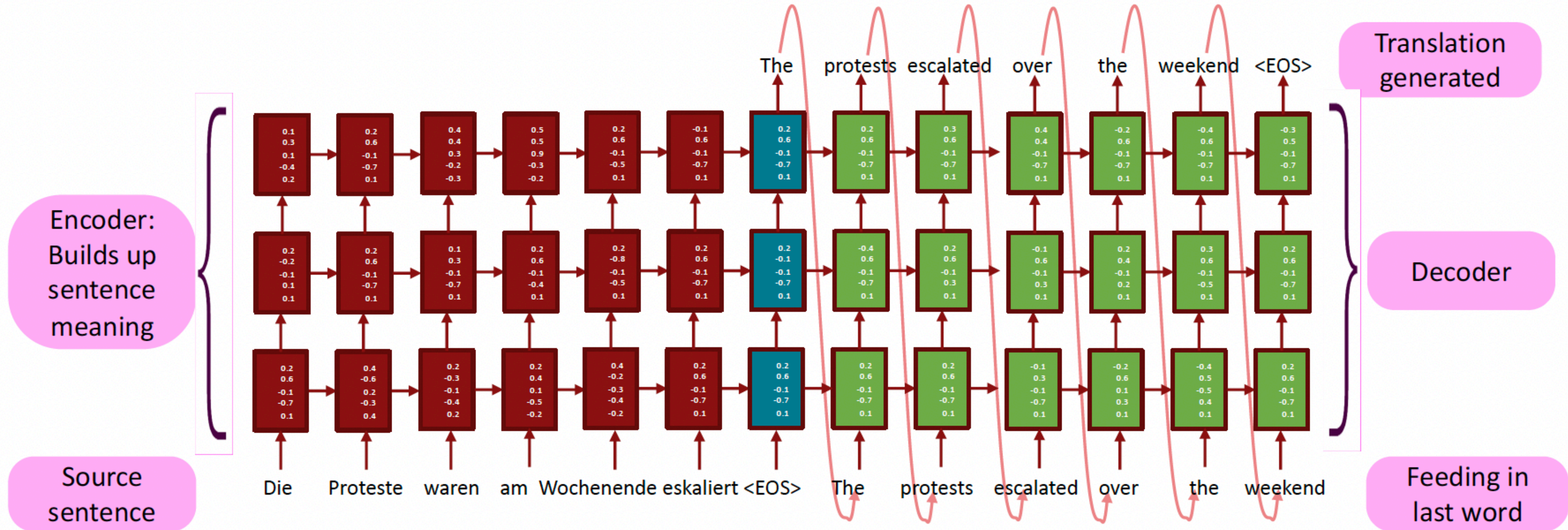
- The decoder is feed with the targeted sentence during training



Multi-layer deep encoder-decoder machine translation net

[Sutskever et al. 2014; Luong et al. 2015]

The hidden states from RNN layer i are the inputs to RNN layer $i+1$



Conditioning =
Bottleneck

- High-performing RNNs are usually multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For example, for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
 - Often 2 layers is a lot better than 1, and 3 might be a little better than 2
 - Usually, skip-connections/dense-connections are needed to train deeper RNNs (e.g., 8 layers)
- Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.
- You will learn about Transformers later; they have a lot of skipping-like connections

Word Embedding

Why We Need Embeddings

- The Problem: Computers work with numbers; humans communicate in words.
- The Traditional Gap: Keyword search relies on exact matches (e.g., 'energy production' vs. 'powerhouse' = NO MATCH).
- The Solution: Embeddings convert text into high-dimensional numerical vectors capturing semantic meaning.
- The Outcome: Distance between vectors indicates conceptual similarity, making search robust.

1-hot Word Representation

- $V = \{a, aaron, \dots, zulu, <UNK>\}$. $|V| \sim 10,000$
- 1-hot representation for V :

man	woman	king	queen	apple	orange
(5391)	(9853)	(4914)	(7157)	(456)	(6257)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$

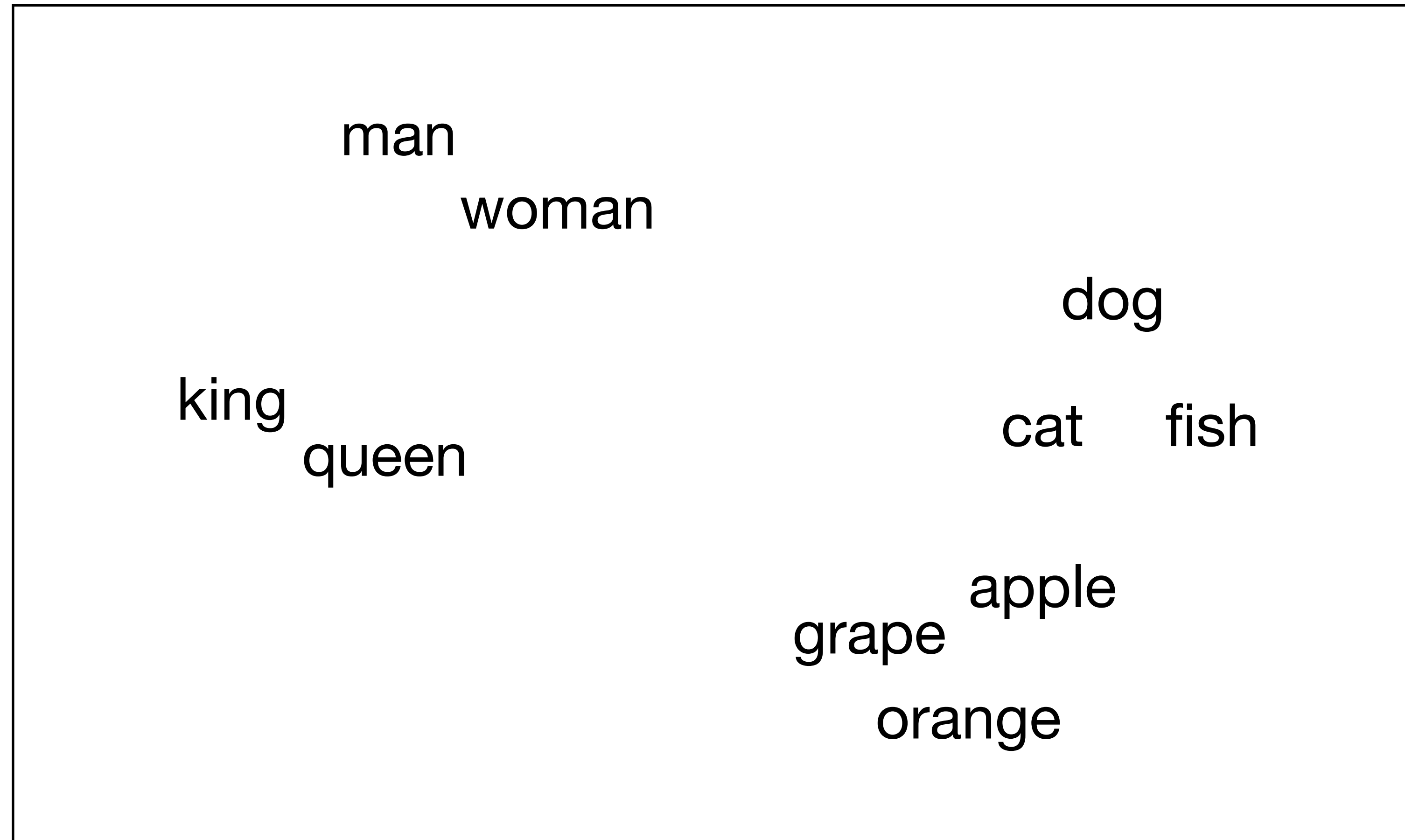
1-hot Representation

- Given a vocabulary V , the i -th word in V is assigned a vector of size $|V|$ such that its i -th entry is 1 and all other entries are 0.
- However, such a 1-hot embedding does not capture semantic relationships, e.g., the representations of “apple” and “orange” should be relatively close.

Featurized representation: word embedding

Features	man	woman	king	queen	apple	orange
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
	...	...	...	...	...	...

Visualizing word embeddings



Analogies

Features	man	woman	king	queen	apple	orange
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
	...	...	...	...	...	...

man → woman

king → ?

$$e_{man} - e_{woman} \approx e_{king} - e_?$$

Analogies

Features	man	woman	king	queen	apple	orange
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
	...	...	...	...	...	...

man → woman

king → ?

$$e_{man} - e_{woman} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$e_{king} - e_{queen} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$e_{man} - e_{woman} \approx e_{king} - e_{?}$$

$$e_{man} - e_{woman} \approx e_{king} - e_{queen}$$

Analogies

Features	man	woman	king	queen	apple	orange
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
	...	...	...	...	...	...

man → woman
king → ? **queen**

$$e_{man} - e_{woman} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$
$$e_{king} - e_{queen} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$e_{man} - e_{woman} \approx e_{king} - e_{?}$$

$$e_{man} - e_{woman} \approx e_{king} - e_{queen}$$

Cosine similarity

$$\textit{sim}(u, v) = \frac{\langle u, v \rangle}{|u| \cdot |v|}$$

Man:Woman as Boy:Girl

Ottawa:Canada as Nairobi:Kenya

Big:Bigger as Tall:Taller

Yen:Japan as Ruble:Russia

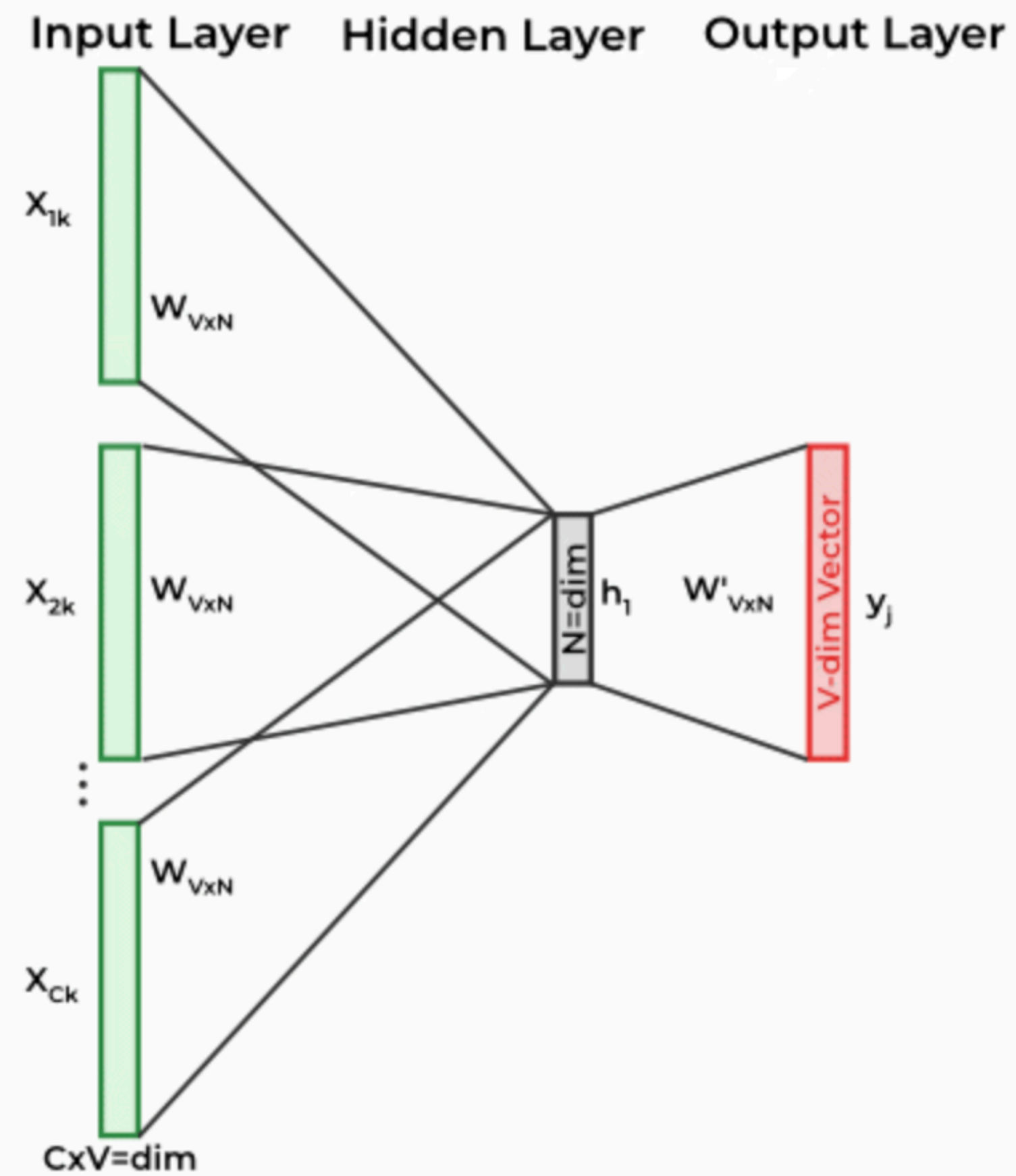
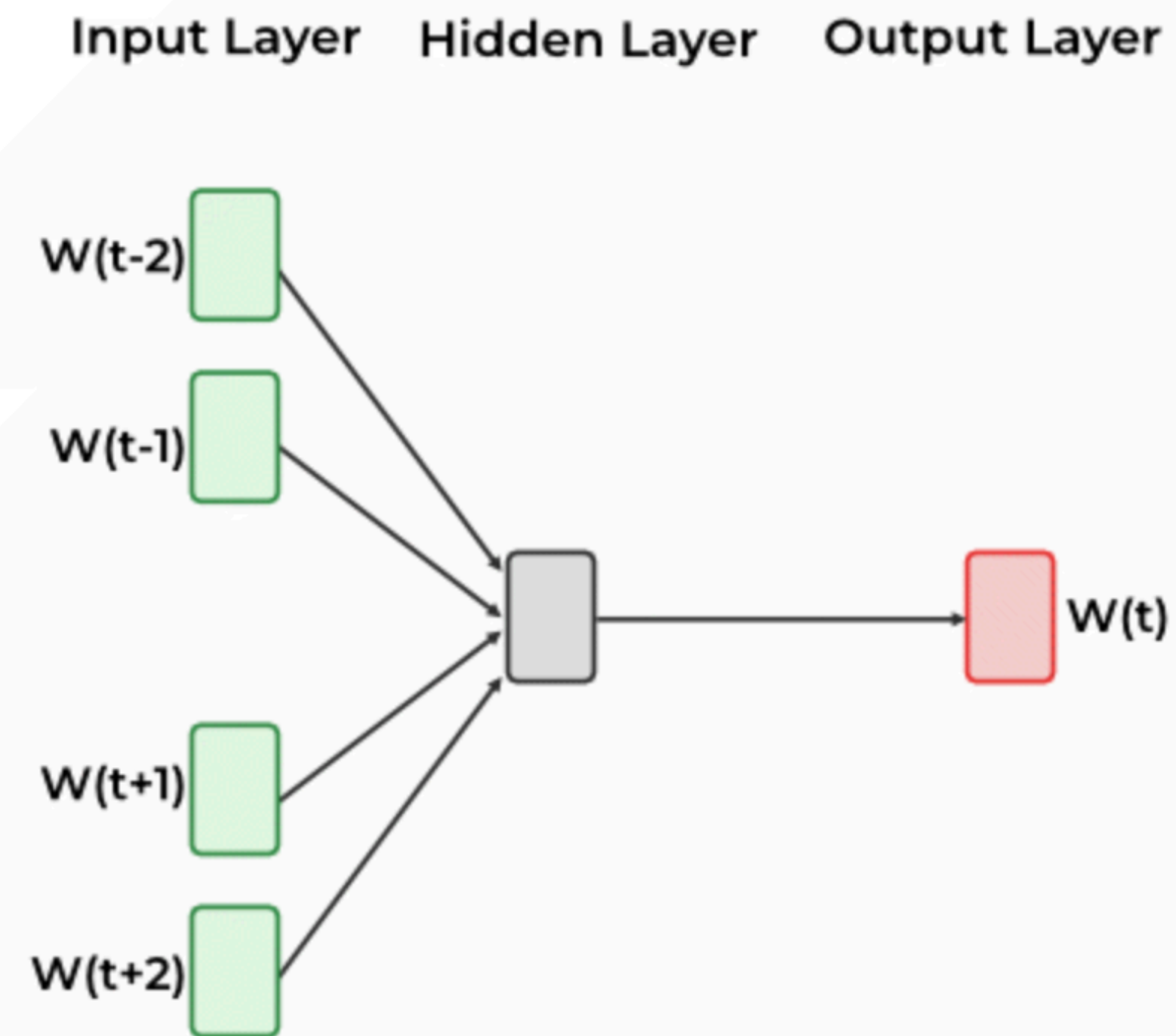
Word2Vec

Uses shallow neural networks such as Skip-gram to learn embeddings by predicting a target word from its context, or vice versa.

Word2Vec

- Developed by researchers at Google (2013).
- Uses a shallow neural network to learn word associations.
- Two main architectures:
 - Continuous Bag of Words (CBOW): Predicts a target word from its surrounding context words.
 - Skip-Gram: Predicts surrounding context words given a single target word.

CONTINUOUS BAG OF WORDS (CBOW)



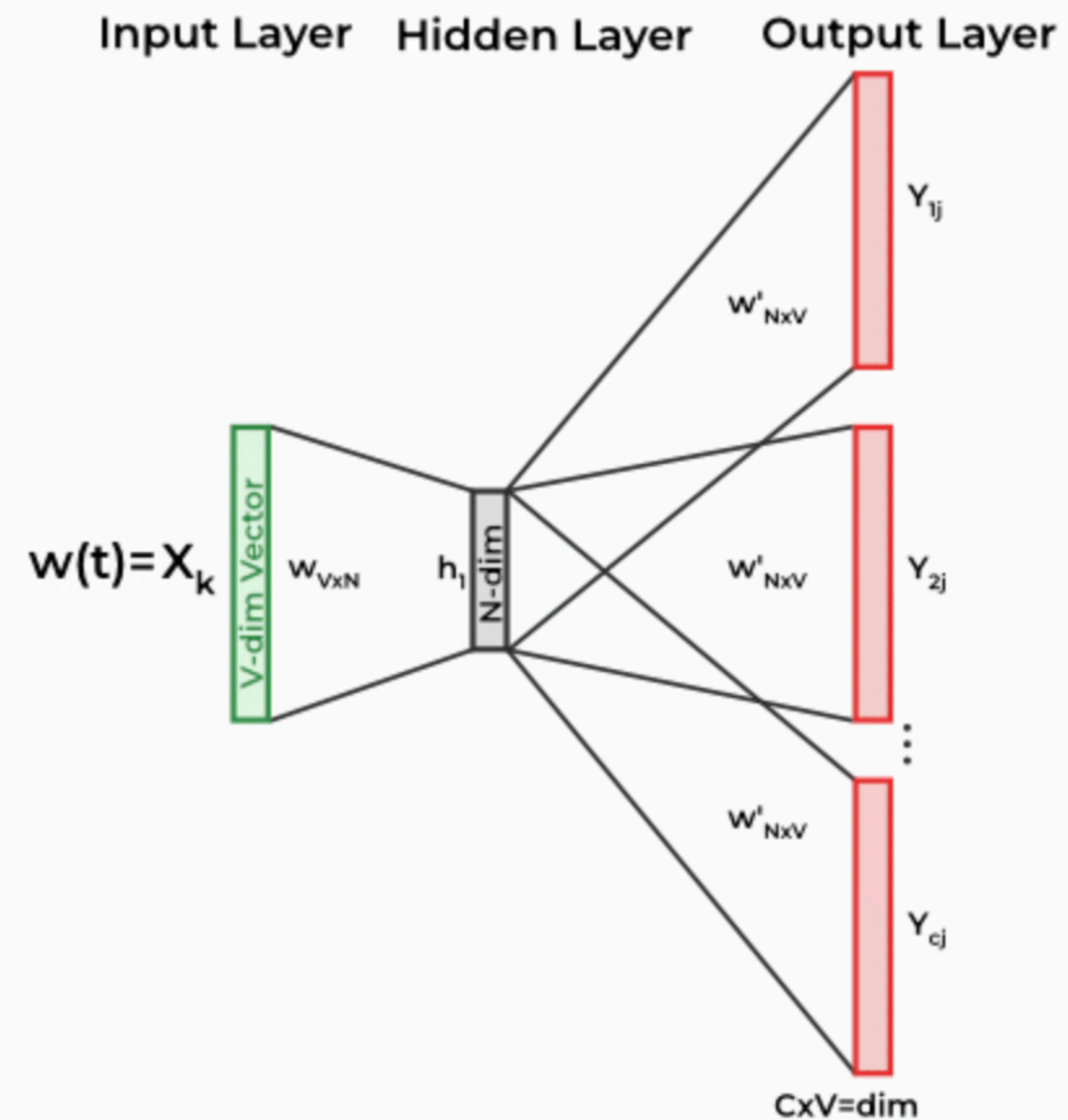
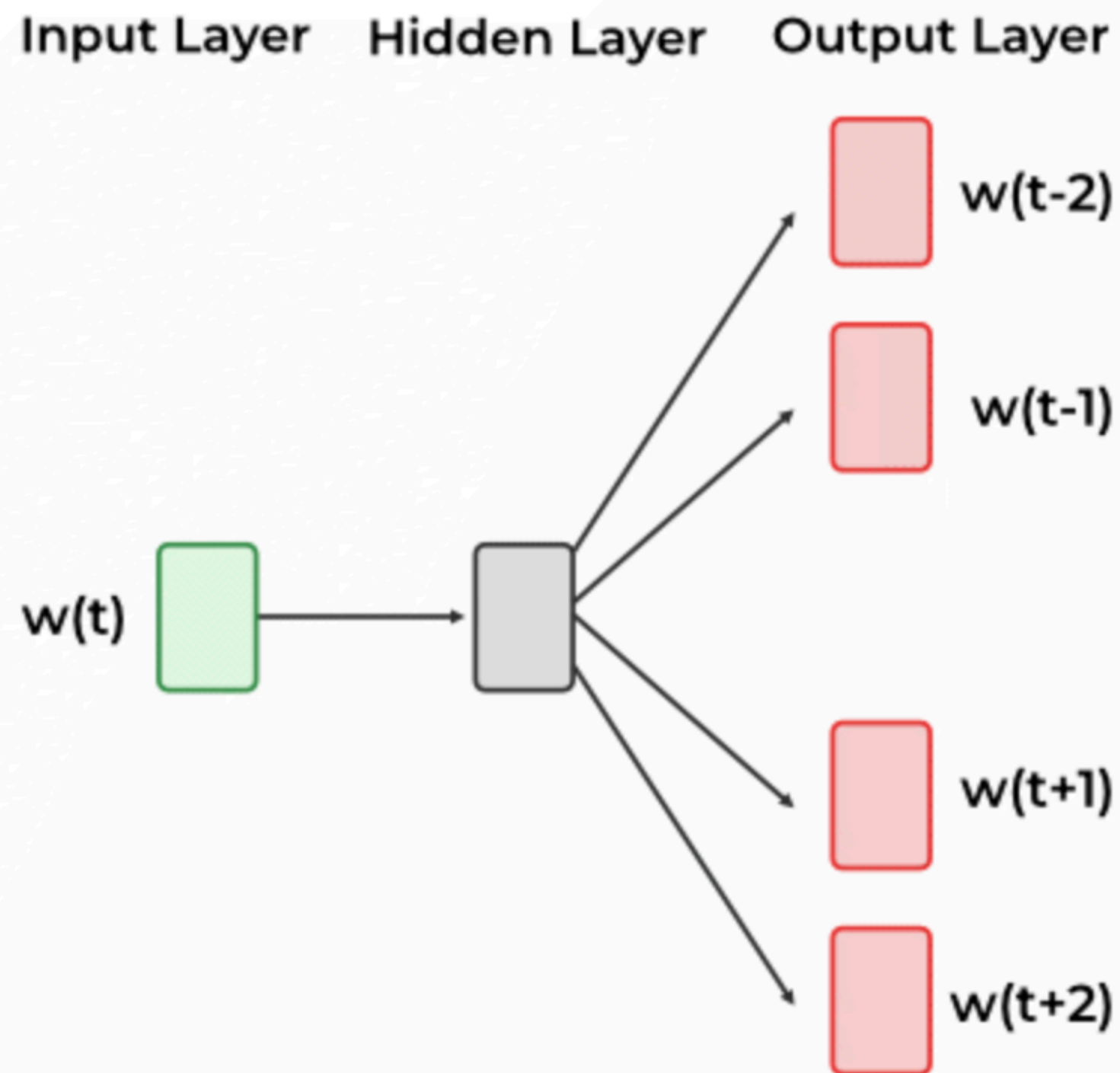
Sentence:

["India", "wins", "next", "world", "cup"]

CBOW Training Example

- Context: ["India", "wins", "next"] → Target: "world"

SKIP-GRAM ARCHITECTURE



Skip-Gram Training Example

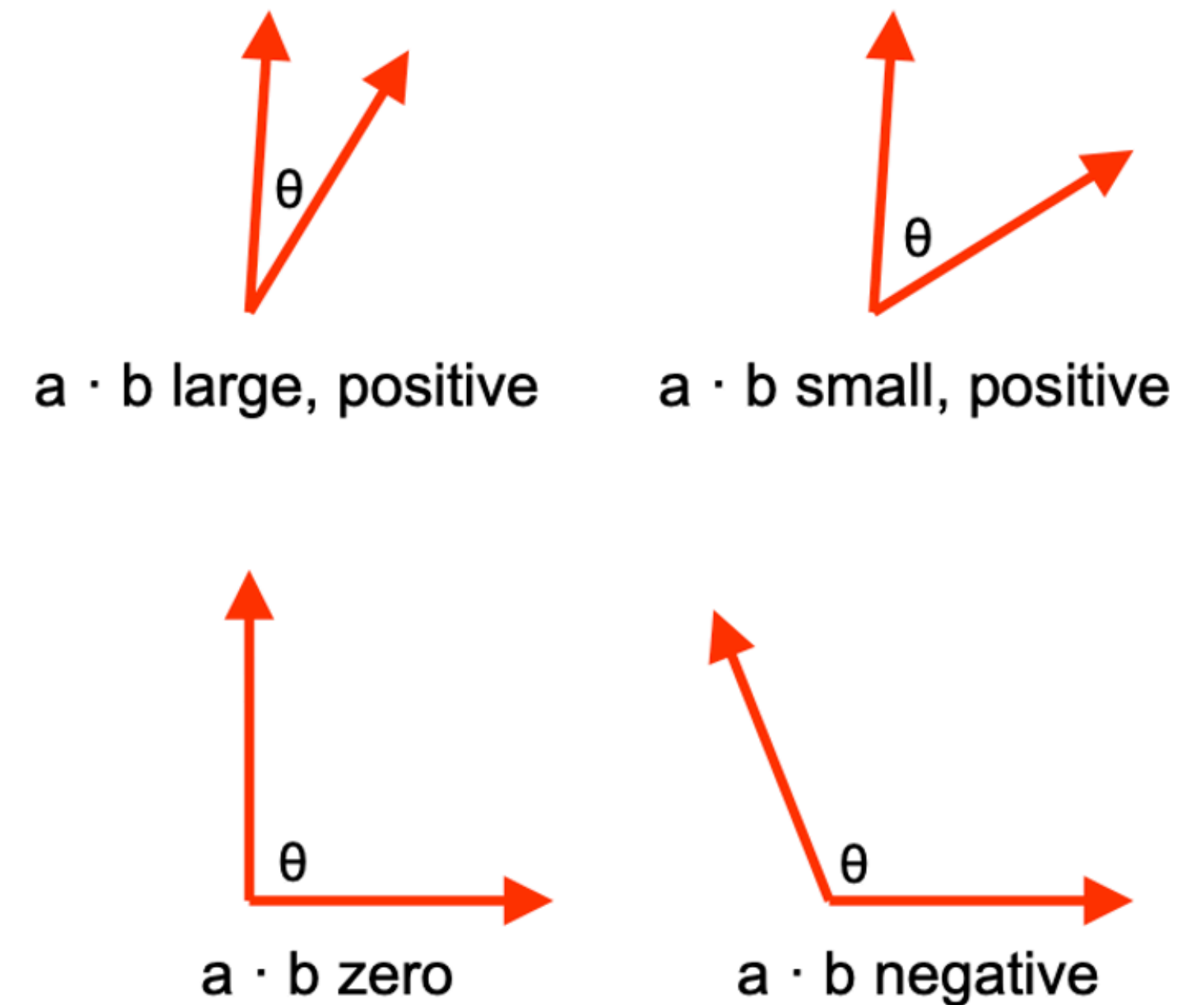
- Target: "India" $\rightarrow$ Context: ["wins", "next", "world"]
- Target: "wins" $\rightarrow$ Context: ["India", "next", "world"]
- Target: "next" $\rightarrow$ Context: ["wins", "world", "cup"]

Negative Sampling

- updates model weights using a small subset of "negative" (incorrect) examples rather than the entire dataset

GloVe (Global Vectors)

- **GloVe** is an unsupervised learning algorithm for word embedding.
- **The Goal:** To capture semantic and syntactic similarity for vectors.
- **Advantage:** Word2Vec relied entirely on local sliding context windows to learn relationships. GloVe is designed to capture both local and global contexts.
- **Intuition:** The meaning of a word is defined by the company it keeps.



Key Advantages and Capabilities

- Why GloVe Became an Industry Standard:
 - Training Efficiency: It trains remarkably fast on massive, web-scale datasets.
 - Linear Substructures: GloVe is famous for producing vector spaces where complex semantic relationships can be solved using simple vector algebra.
 - The classic example: $\text{vector}(\text{"King"}) - \text{vector}(\text{"Man"}) \approx \text{vector}(\text{"Queen"}) - \text{vector}(\text{"Woman"})$
 - Holds true for capitals (Paris - France $\approx$ Rome - Italy) and verb tenses (Walking - Walk $\approx$ Swimming - Swim).
 - Versatility: Pre-trained GloVe embeddings can be easily used in downstream ML tasks

Various Word Embeddings

- Word-Level Models (Basic): Word2Vec, GloVe. Assigns one embedding per word, ignoring context.
- Contextual Models (Better): BERT, RoBERTa. Embeddings adapt based on surrounding words.
- Sentence/Document Models (Best for RAG): Sentence-BERT, OpenAI Embeddings.
 - Specially trained to embed entire sentences or paragraphs.
 - Crucial for retrieving relevant document chunks in RAG architectures.